

Arca

Arquitetura do banco de dados

Como a Arca organiza e grava os dados em disco

LSM-Tree, formatos binários do WAL, das SSTables e do MANIFEST, codificação de chaves e linhas, compactação, recuperação e o modelo lógico das tabelas. É a arquitetura que a aplicação segue e que qualquer extensão deve respeitar.



Versão 1.0.0 · LSM-Tree-Database-Server · Copyright (©) 2026 Erik Pimenta · Licença MIT

Sumário

1 Visão da arquitetura

2 Arquivos no diretório

3 Chave interna e sequência

4 Log de escrita (WAL)

5 Memtable

6 SSTable

7 MANIFEST e versões

8 Compactação

9 Recuperação após queda

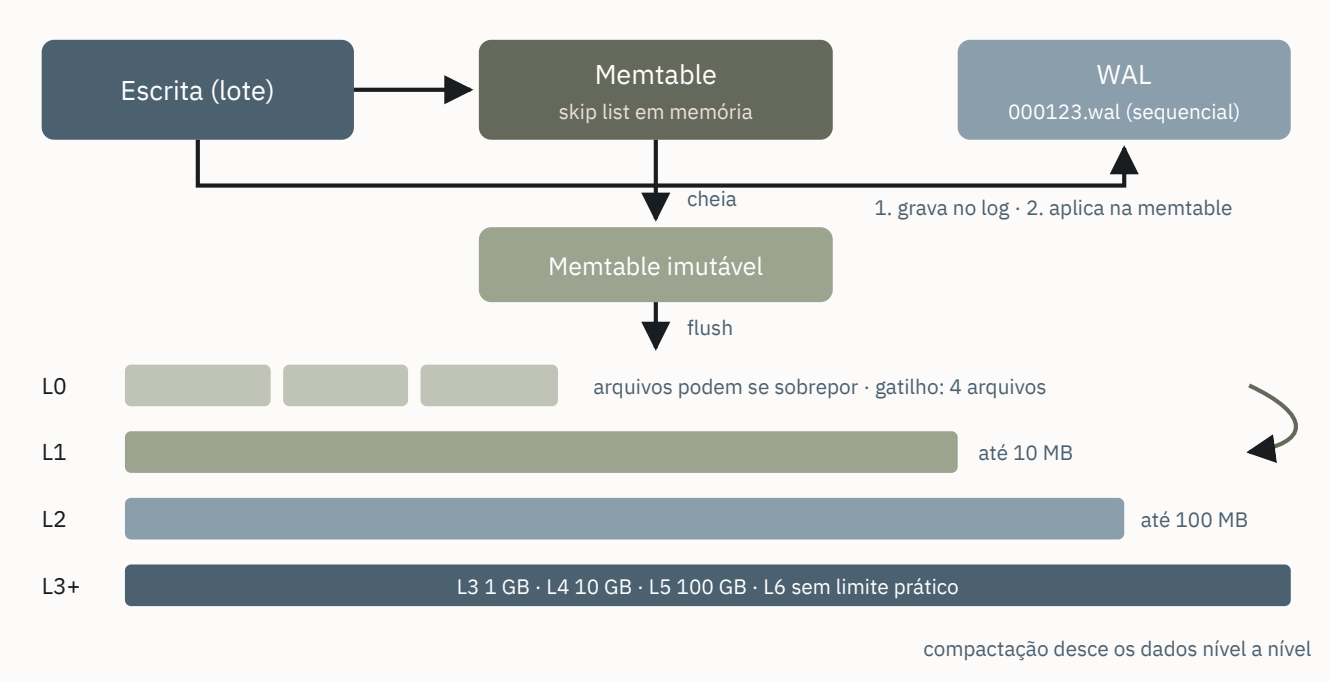
10 Modelo lógico: chaves, tabelas e índices

11 Concorrência

12 Preparando o armazenamento

1 Visão da arquitetura

A Arca usa uma **Log-Structured Merge-Tree**. Nenhum dado é alterado no lugar: cada mudança é acrescentada e versões antigas somem quando os arquivos são mesclados. A figura mostra o caminho de uma escrita.



Escritas sequenciais no WAL e na memtable; arquivos ordenados e imutáveis em níveis

Camada	Onde fica	Papel
WAL	`NNNNNN.wal`	Garante que nada confirmado se perca antes de virar SSTable
Memtable	memória	Skip list ordenada com as escritas recentes; leitura sem trava
Memtable imutável	memória	A memtable cheia, aguardando a gravação em disco
SSTables	`NNNNNN.sst`	Arquivos ordenados, imutáveis, com índice, Bloom e checksums
MANIFEST	`MANIFEST-NNNNNN`	Log das versões: quais SSTables existem em cada nível
CURRENT	`CURRENT`	Nome do MANIFEST em uso, trocado de forma atômica

2 Arquivos no diretório

Cada banco é uma pasta. Os números de arquivo vêm de um único contador, então nunca se repetem.

Arquivo	Exemplo	Conteúdo	Quando é apagado
CURRENT	CURRENT	Uma linha com o nome do MANIFEST atual	Nunca (é substituído por renomeação)
LOCK	LOCK	Trava do sistema operacional (fcntl ou abertura exclusiva no Windows)	Nunca
MANIFEST	MANIFEST-000005	Registros de VersionEdit no formato do log	Quando um MANIFEST mais novo assume
WAL	000007.wal	Lotes de escrita (WriteBatch) no formato do log	Depois que a memtable dele vira SSTable
SSTable	000012.sst	Tabela ordenada e imutável	Quando nenhuma versão viva a referencia
Temporário	000005.tmp	Arquivo do CURRENT antes da renomeação	Logo após a troca

> Um arquivo só é removido quando não aparece em nenhuma versão ainda em uso. Iteradores e snapshots seguram a versão que estavam lendo, então uma varredura longa nunca perde arquivos no meio do caminho.

3 Chave interna e sequência

Toda escrita recebe um número de sequência de 56 bits que só cresce. A chave guardada no motor é a **chave interna**: a chave do usuário seguida de 8 bytes que juntam a sequência e o tipo da operação.



8 bytes little-endian: (sequência << 8) | tipo

Tipo	Valor	Significado
kTypeDeletion	0	Lápide: a chave foi apagada nesta sequência
kTypeValue	1	Valor gravado nesta sequência

A ordenação compara primeiro a chave do usuário em bytes e, para a mesma chave, a sequência **decrescente**. Assim a versão mais nova aparece primeiro e uma busca por (chave, sequência S) cai exatamente na versão mais nova visível em S. É isso que dá snapshots MVCC de graça.

4 Log de escrita (WAL)

O log é dividido em blocos de 32 KB. Cada registro lógico é quebrado em fragmentos físicos que nunca atravessam um bloco; um resto de bloco menor que o cabeçalho é preenchido com zeros.

Campo	Tamanho	Descrição
checksum	4 bytes	CRC32C mascarado do tipo + dados
tamanho	2 bytes	Bytes de dados deste fragmento
tipo	1 byte	FULL (1), FIRST (2), MIDDLE (3), LAST (4)
dados	variável	Pedaço do registro

Conteúdo de um registro: WriteBatch

Campo	Codificação
sequência inicial	fixo de 8 bytes
quantidade de operações	fixo de 4 bytes
cada operação	tipo (1 byte) + chave com prefixo de tamanho (varint) + valor com prefixo de tamanho, se for escrita

A operação i do lote recebe a sequência inicial + i. Um registro com checksum errado ou cortado no fim do arquivo é descartado na recuperação; o restante do log continua valendo. Com `paranoid-checks` ligado, qualquer corrupção impede a abertura.

Group commit

Escritores concorrentes entram numa fila. O primeiro vira líder, junta os lotes de quem está esperando (até 1 MB), grava um único registro e, se preciso, faz um único `fsync`. Os demais acordam com o resultado pronto. Com `appendfsync always`, isso divide o custo do fsync entre todos.

5 Memtable

A memtable é uma **skip list** com até 12 níveis (probabilidade 1/4 de subir), alocada numa arena de blocos de 4 KB. Só um escritor insere por vez (o líder do group commit); leitores percorrem a lista sem trava, protegidos por ponteiros atômicos com ordem acquire/release.

Parte da entrada	Codificação
tamanho da chave interna	varint32
chave interna	chave do usuário + 8 bytes de sequência e tipo
tamanho do valor	varint32
valor	bytes

Quando o uso da arena passa de `write-buffer-size`, a memtable vira imutável, um WAL novo começa e a thread de fundo grava a SSTable. Se ainda houver uma memtable imutável pendente, novas escritas esperam (travamento de escrita).

6 SSTable



cada bloco termina com 1 byte de compressão (0 = nenhuma) e 4 bytes de CRC32C
rodapé de 68 bytes: referências (offset, tamanho) de Bloom, propriedades e índice + número mágico 0x4152434153535442

Layout de um arquivo .sst

Bloco de dados

Blocos de cerca de 4 KB com entradas ordenadas. Para economizar espaço, cada chave guarda só a parte que difere da anterior; a cada 16 entradas há um **ponto de reinício** com a chave completa, usado na busca binária dentro do bloco.

Campo da entrada	Codificação
bytes compartilhados com a chave anterior	varint32
bytes novos da chave	varint32
tamanho do valor	varint32
parte nova da chave	bytes
valor	bytes
(fim do bloco) pontos de reinício	fixo de 4 bytes cada + quantidade em 4 bytes

Índice, Bloom e propriedades

- **Índice:** um bloco com uma entrada por bloco de dados; a chave é a última chave interna do bloco e o valor é (offset, tamanho) em varint.
- **Filtro de Bloom:** um filtro por arquivo sobre as chaves do usuário, com 10 bits por chave e 6 funções de hash (dupla dispersão), cerca de 1% de falso positivo. Uma leitura pontual consulta o filtro antes de tocar no disco.
- **Propriedades:** entradas, lápides, blocos, bytes de chave e de valor, menor e maior sequência, data de criação e bits do Bloom. O `arca-inspect` mostra todas.

Cache

Blocos lidos vão para um cache LRU dividido em 16 fatias, cada uma com seu mutex, limitado por `block-cache-size`. Arquivos abertos ficam num cache de tabelas protegido por `shared_mutex`: consultas usam trava compartilhada e só a abertura de um arquivo novo usa trava exclusiva.

7 MANIFEST e versões

Uma **versão** é a lista de SSTables de cada nível. Versões são imutáveis e compartilhadas por ponteiro com contagem de referências. Toda mudança (flush, compactação) é descrita por um **VersionEdit**, gravado no MANIFEST com `fsync` antes de a nova versão entrar em uso.

Marcador	Campo	Codificação
2	número do log em uso	varint64
3	próximo número de arquivo	varint64
4	última sequência	varint64
5	ponteiro de compactação	nível + chave interna
6	arquivo removido	nível + número
7	arquivo adicionado	nível, número, tamanho, entradas, lápides, menor e maior chave

Ao abrir o banco, um MANIFEST novo é criado com o retrato completo da versão atual; depois o CURRENT passa a apontar para ele por renomeação atômica e o antigo é apagado. O `arca-inspect manifest` mostra cada edição em texto.

8 Compactação

Nível	Gatilho	Regra
L0	4 arquivos (pontuação = arquivos ÷ 4)	Arquivos se sobrepõem; todos os que tocam o intervalo entram juntos
L1	mais de 10 MB	Arquivos não se sobrepõem; um arquivo por vez, em rodízio pelo ponteiro de compactação
Lk (k ≥ 2)	mais de 10 MB × 10^{k-1}	Igual ao L1

A thread de fundo escolhe o nível com maior pontuação maior que 1, junta os arquivos escolhidos com os que se sobrepõem no nível de baixo e grava SSTables novas de até `max-file-size`, cortando também quando a saída se sobreporia demais ao nível seguinte. Um arquivo que não se sobrepõe a nada é só movido de nível, sem reescrita.

O que é descartado

1. Versões antigas de uma chave cuja versão mais nova já é visível para todos os snapshots abertos.
2. Lápides com sequência visível para todos, quando nenhum nível mais profundo pode ter a chave.

Snapshots seguram versões: a menor sequência de snapshot aberto é o limite abaixo do qual a compactação pode limpar. Os travamentos de escrita começam com 8 arquivos no L0 (pausa de 1 ms por escrita) e as escritas param com 12, até a compactação alcançar.

`ARCA.COMPACT` grava a memtable e compacta cada nível inteiro para o de baixo até o nível mais profundo que já tem dados, o que remove lápides e versões antigas de todo o banco.

9 Recuperação após queda

1. Trava o diretório pelo arquivo LOCK.
2. Lê o CURRENT e reaplica cada VersionEdit do MANIFEST, reconstruindo a versão, o próximo número de arquivo e a última sequência.
3. Confere que todas as SSTables citadas existem; se faltar alguma, a abertura falha e informa qual.
4. Reaplica, em ordem, os WALs com número maior ou igual ao log registrado. Fragmentos com checksum errado ou cortados no fim são descartados.
5. Grava o que foi reaplicado como SSTables do L0, cria um WAL novo e um MANIFEST novo.
6. Apaga logs, MANIFESTs e SSTables que não pertencem a nenhuma versão viva.

> Como as SSTables são imutáveis e só passam a valer depois do `fsync` do MANIFEST, uma queda no meio de um flush ou de uma compactação deixa apenas arquivos órfãos, que são apagados na próxima abertura.

10 Modelo lógico: chaves, tabelas e índices

Chave-valor e SQL convivem no mesmo espaço de chaves, separados pelo primeiro byte. Como a ordenação é por bytes, cada espaço ocupa uma faixa contínua e pode ser varrido sem tocar nos outros.

Prefixo	Espaço	Chave	Valor
0x01	chave-valor	0x01 + chave	flag (1 byte) + [expiração em ms, 8 bytes] + valor
0x02	catálogo	0x02 + nome da tabela	definição: id, chave primária, colunas, índices
0x03	linhas	0x03 + id da tabela (4 bytes big-endian) + chave primária ordenável	linha codificada
0x04	índices	0x04 + id do índice + marcador de nulo + valor ordenável + chave primária	vazio
0x05	contadores	0x05 + nome	inteiro de 8 bytes (próximo id, autoincremento, linhas)
0x06	expiração	0x06 + instante (8 bytes big-endian) + chave	vazio

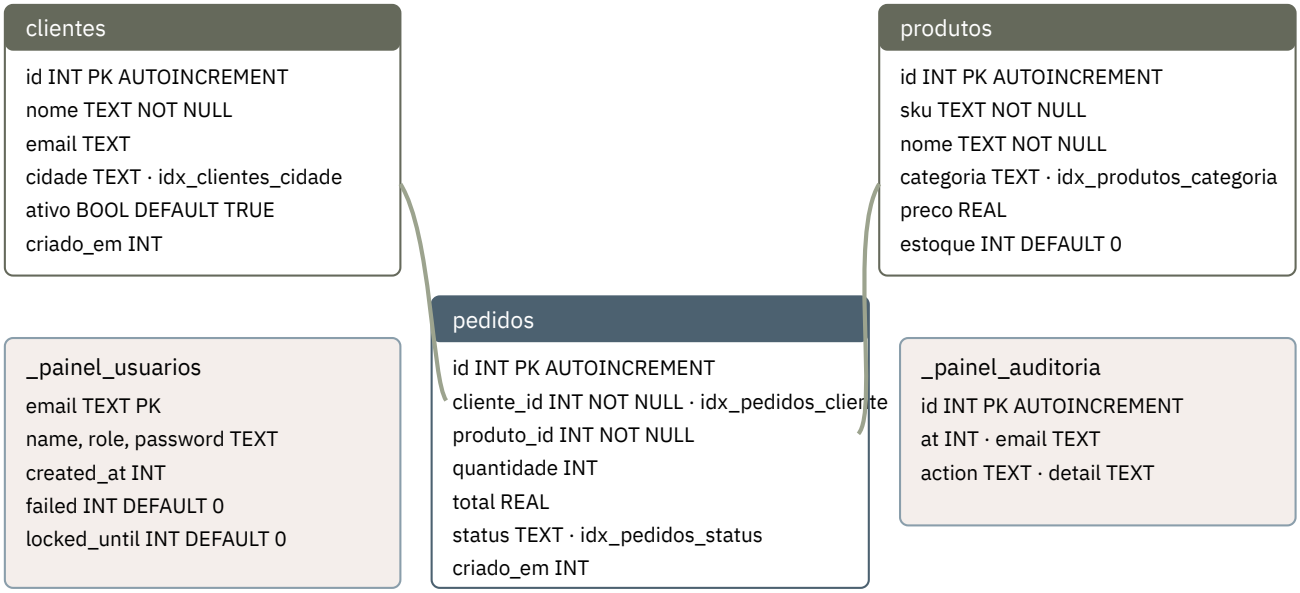
Codificação ordenável

Tipo	Bytes	Por que ordena
INT	8 bytes big-endian com o bit de sinal invertido	Negativos ficam antes dos positivos
REAL	8 bytes IEEE 754: positivos com o sinal ligado, negativos com todos os bits invertidos	A ordem dos bytes segue a ordem numérica
TEXT	bytes com 0x00 trocado por 0x00 0xFF e terminador 0x00 0x01	Prefixo vem antes da palavra maior
BOOL	1 byte (0 ou 1)	FALSE antes de TRUE

Linha

Quantidade de colunas em varint e, para cada coluna, um marcador de tipo (0 nulo, 1 inteiro em zigzag varint, 2 real de 8 bytes, 3 texto com tamanho, 4 booleano). Linhas antigas com menos colunas leem as que faltam como NULL, o que torna `ALTER TABLE ADD COLUMN` barato.

Tabelas criadas pela aplicação



Tabelas de exemplo (criadas com ARCA_SEED=true) e tabelas de sistema do painel

As relações entre `pedidos`, `clientes` e `produtos` são lógicas: a Arca não impõe chave estrangeira, e as consultas usam os índices `idx_pedidos_cliente` e `idx_pedidos_status`. Tabelas iniciadas por `_` são de sistema. As sessões do painel ficam no espaço chave-valor como `_painel:sessao:<sha256 do token>`, com TTL.

11 Concorrência

Recurso	Mecanismo	Quem disputa
Fila de escrita	`std::mutex` + uma `condition_variable` por escritor (group commit)	Todos os comandos que gravam
Arquivo do WAL	`std::mutex` próprio	Líder da fila, thread de fsync, troca de memtable
Versão atual (SuperVersion)	`std::shared_mutex`: leitura compartilhada, troca exclusiva	Leituras e a thread de fundo
Memtable	Skip list com ponteiros atômicos (acquire/release)	Um escritor e vários leitores sem trava
Cache de blocos	16 fatias, cada uma com `std::mutex`	Leituras
Cache de tabelas	`std::shared_mutex`	Leituras e aberturas de arquivo
Catálogo SQL	`std::shared_mutex` global + um por tabela	DDL exclusivo; SELECT compartilhado; INSERT/UPDATE/DELETE exclusivos por tabela
Chaves	1024 mutex em faixas por hash da chave	INCR, APPEND, SET NX/XX, expiração
MULTI/EXEC	`std::shared_mutex`: comandos compartilham, EXEC é exclusivo	Transações
Rede	Uma thread por laço de eventos; fila com mutex para conexões novas e respostas dos workers	Laços e workers

12 Preparando o armazenamento

A Arca não precisa de um servidor de banco à parte nem de script de criação: o diretório e os arquivos são criados na primeira abertura, e as tabelas surgem com `CREATE TABLE`. O que vale preparar:

- **Disco:** SSD é recomendado. Reserve pelo menos o dobro do tamanho dos dados, porque a compactação grava as SSTables novas antes de apagar as antigas.
- **Sistema de arquivos:** um com `fsync` confiável (ext4, xfs, APFS, NTFS). Evite pastas de rede compartilhadas.
- **Docker:** os dados ficam no volume `dados` (`/dados/banco` e `/dados/checkpoints`). O contêiner roda como usuário sem privilégios (uid 10001).
- **Permissões:** o usuário do processo precisa ler e gravar a pasta do banco e a pasta `checkpoints` ao lado dela.
- **Backup:** agende `ARCA.CHECKPOINT` e copie a pasta gerada para outro lugar; restaurar é apontar `dir` para ela.
- **Migração entre máquinas:** os formatos têm ordem de bytes definida e não dependem da plataforma; a pasta pode ser copiada entre Linux, macOS e Windows com o servidor parado.

Verificação	Comando
Checksums de todas as SSTables	`arca-inspect check dados`
Versão atual e arquivos por nível	`arca-inspect manifest dados/MANIFEST-NNNNNN`
Lotes ainda não gravados em SSTable	`arca-inspect wal dados/NNNNNN.wal`
Estado ao vivo	`arca-cli ARCA.STATS` ou a tela Armazenamento do painel